

《操作系统》

大作业

题目： 在 Linux 操作系统上实现基于位示图简单的文件系统
姓名：
学号：
班级： 22 计算机科学与技术 1 班
指导老师： 汤峰老师
日期： 2025 年 05 月

小 题 1 ： 模 拟 位 示 图 管 理 磁 盘 空 间 的 分 配 与 回 收

一、 题 目 要 求

- (1)
 假
 设
 现
 在
 有
 一
 个
 硬
 盘
 ，
 有
 4
 个
 磁
 头
 ，
 40
 个
 柱
 面
 ，
 1
 个
 磁
 头
 和
 1
 个
 柱
 面
 划
 分
 出
 8
 个
 扇
 区
 。
 磁
 盘
 的
 空
 间
 使
 用
 情
 况
 由
 位
 示
 图
 表
 示
 。
 位
 示
 图
 的
 每
 一
 位
 对
 应
 一
 个
 磁
 盘
 块
 ，
 1
 表
 示
 占
 用
 ，
 0
 表
 示
 空
 闲
 。
- (2)
 申
 请
 一
 块
 磁
 盘
 块
 时
 ，
 由
 分
 配
 程
 序
 查
 位
 示
 图
 ，
 找
 到
 一
 个
 为
 0
 的
 位
 ，
 并
 计
 算
 磁
 盘
 的
 物
 理
 地
 址
 （
 即
 求
 出
 磁
 盘
 块
 对
 应
 的
 柱
 面
 号
 、
 磁
 头
 号
 和
 扇
 区
 号
 ）
 。
- (3)
 当
 释
 放
 一
 个
 相
 对
 物
 理
 块
 时
 ，
 运
 行
 回
 收
 程
 序
 ，
 计
 算
 该
 磁
 盘
 块
 在
 位
 示
 图
 中
 的
 位
 置
 ，
 在
 把
 相
 应
 的
 位
 清
 零
 。
- (4)
 设
 计
 一
 个
 磁
 盘
 块
 分
 配
 程
 序
 。
 输
 入
 需
 要
 申
 请
 的
 磁
 盘
 块
 数
 ，
 运
 行
 分
 配
 程
 序
 后
 返
 回
 这
 些
 磁
 盘
 块
 的
 物
 理
 地
 址
 （
 柱
 面
 号
 、
 磁
 头
 号
 和
 扇
 区
 号
 ）
 ，
 并
 且
 打
 印
 位
 示
 图
 。
- (5)
 设
 计
 一
 个
 磁
 盘
 块
 回
 收
 程
 序
 ，
 输
 入
 一
 系
 列
 磁
 盘
 块
 的
 物
 理
 地
 址
 ，
 在
 位
 示
 图
 上
 对
 应
 的
 位
 置
 清
 零
 ，
 表
 示
 释
 放
 了
 该
 磁
 盘
 块
 。
 并
 且
 打
 印
 位
 示
 图
 。

二、需求分析

2 个主要功能，磁盘块分配和磁盘块回收，以及显示磁盘块的位示图。
以及显示、重置位示图的函数。

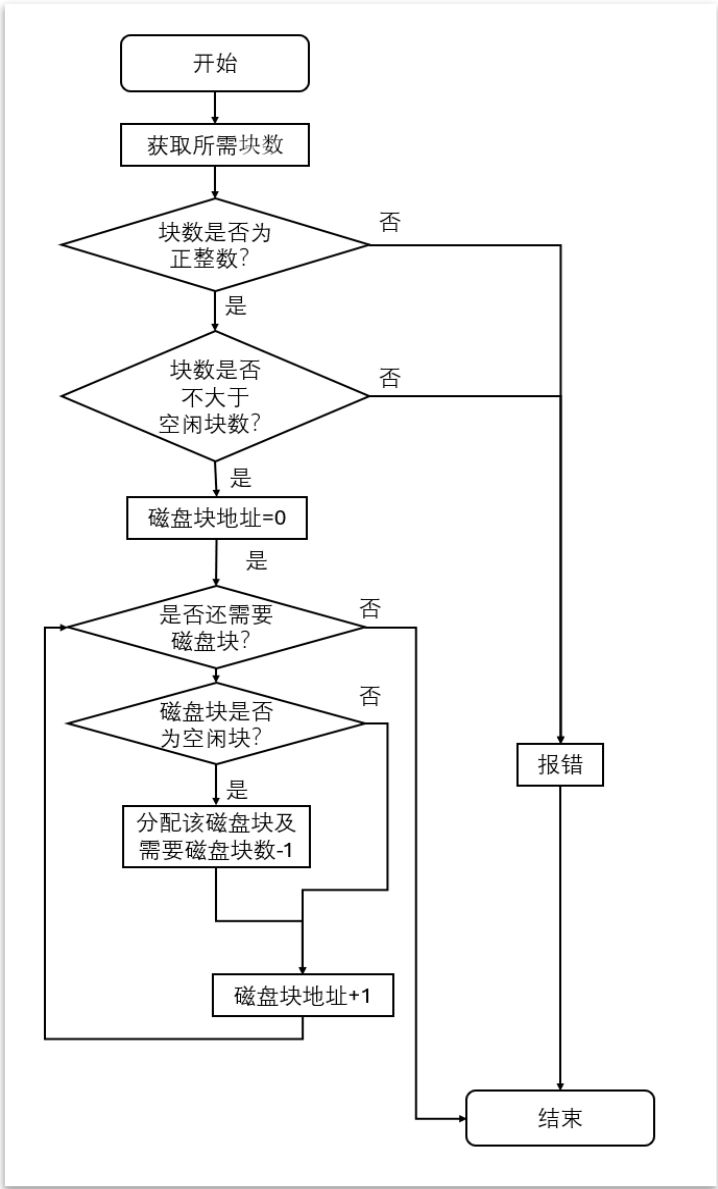


图 1.1 磁盘块分配流程图

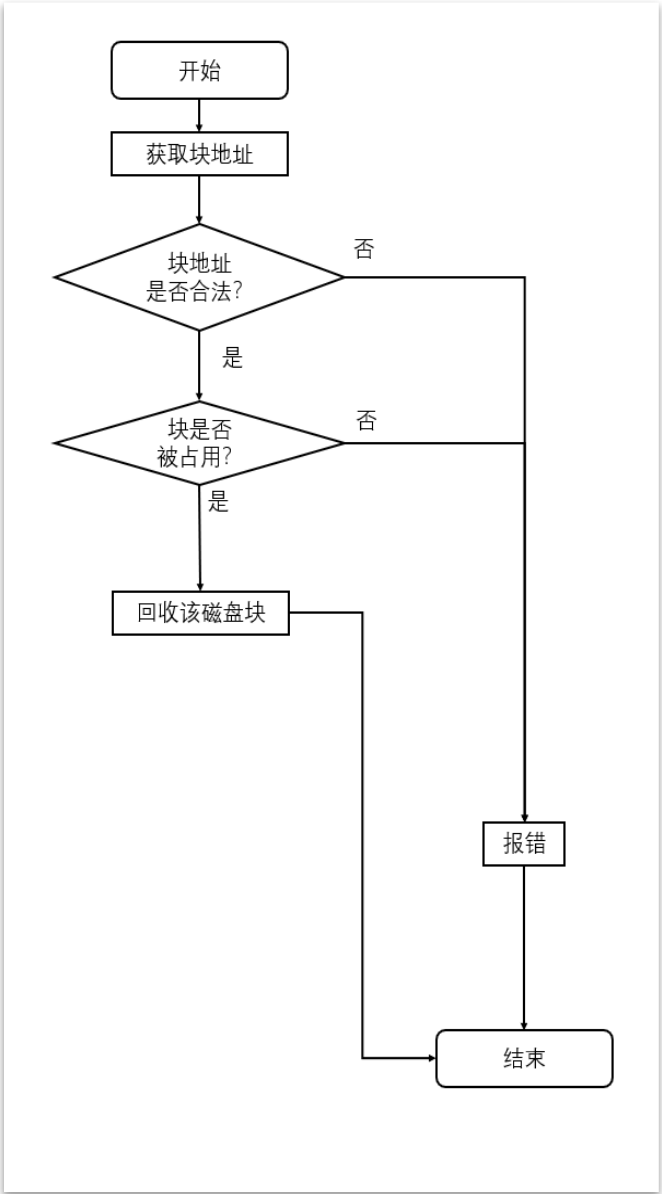


图 1.2 磁盘块回收流程图

三、实验环境

Ubuntu 14.04 LTS
gcc version 4.8.4
编译标准：C99

四、源码注解

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define ROWS 40
#define COLS 32
#define TOTAL_BLOCKS 1280

int bitmap[ROWS][COLS]; // 初始化二维阵列 bitmap 用于保存位示图，初始全为空闲

int cZero = 0; // 磁盘块为空闲的计数器

void resetRandom() {
    /*
     * 重置随机位示图，初始化位示图内容
     */
    srand(time(NULL)); // 用当前时间设置随机种子
    cZero = 0; // 初始化空闲块计数器

    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) { // 遍历位示图
            bitmap[i][j] = rand() % 2; // 随机设置每个块的状态 0/1
            if (bitmap[i][j] == 0) { // 如果是空闲块，计数器加一
                cZero++;
            }
        }
    }
    printf("已重置位示图，free block: %d\n", cZero);
}

void allocate(int num) {
    /*
     * 申请指定数量的空闲磁盘块
     * 参数：
     * num: 还需要的磁盘块数量
     */
    for (int i = 0; i < ROWS && num > 0; i++) {
        for (int j = 0; j < COLS && num > 0; j++) { // 遍历位示图
            if (bitmap[i][j] == 0) { // 如果找到空闲块
                bitmap[i][j] = 1; // 标记为已占用
                // start 计算物理地址
                int blockNum = i * COLS + j;
                int cylinder = blockNum / (4 * 8);
                int temp = blockNum % (4 * 8);
                int head = temp / 8;
                int sector = temp % 8;
                // end 计算物理地址

                printf("申请块地址 %d (柱面:%d 磁头:%d 扇区:%d)\n", blockNum, cylinder, head, sector);

                num--; // 所需磁盘块数量-1
                cZero--; // 空闲块计数器-1
            }
        }
    }
}
```

```

    printf("申请完成，free block: %d\n", cZero);
}

void reclaim(int blockNum) {
    /*
    *回收指定块号的磁盘块
    *参数：
    *blockNum: 需要回收的块号
    */

    int i = blockNum / COLS; //计算索引值
    int j = blockNum % COLS; //计算索引值

    if (bitmap[i][j] == 0) { //检查指定磁盘块是否为空闲块，如是，报错返回
        printf("出错：该磁盘块为空闲块\n");
        return;
    }

    bitmap[i][j] = 0; //如果指定磁盘块为占用块，回收该块
    cZero++; //空闲磁盘块数量+1

    //start 计算物理地址
    int cylinder = blockNum / (4 * 8);
    int temp = blockNum % (4 * 8);
    int head = temp / 8;
    int sector = temp % 8;
    //end 计算物理地址

    printf("回收块地址 %d (柱面:%d 磁头:%d 扇区:%d)\n", blockNum, cylinder, head, sector);
    printf("free block: %d\n", cZero);
}

void displayMatrix() {
    /*
    *显示当前的位示图
    */
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) { //遍历位示图
            printf("%d ", bitmap[i][j]); //输出位示图
        }
        printf("\n");
    }
    printf("free block: %d\n", cZero);
}

int main() {
    int op; //初始化 op 变量用于记录用户操作输入
    resetRandom(); //执行 resetRandom 函数随机化设置位示图

    while (1) {
        printf("\n=== 选择操作 ===\n");
        printf("0. 显示位示图\n");
        printf("1. 申请块\n");
        printf("2. 回收块\n");
        printf("3. 重置位示图\n");
        printf("请输入操作: ");
    }
}

```

```

if (scanf("%d", &op) != 1) {
    printf("输入错误\n");
    while (getchar() != '\n'); // 清除输入缓冲区
    continue;
}

switch (op) {
    case 0://显示位示图
        displayMatrix();
        break;
    case 1://申请块
        {int n;
        printf("请输入申请块数量: ");
        if (scanf("%d", &n) != 1 || n <= 0) {//判断申请块数量是否合法
            printf("出错: 申请块数非正整数\n");
            while (getchar() != '\n');
            break;
        }
        if (n > cZero) {//判断申请块数量是否大于空闲块
            printf("出错: 申请块数 > 空闲块数\n");
            break;
        }
        allocate(n);
        break;}

    case 2://回收块
        {int addr;
        printf("请输入块地址: ");
        if (scanf("%d", &addr) != 1 || addr < 0 || addr >= TOTAL_BLOCKS) {//判断地址是否合法
            printf("出错: 块地址非法, 应在 0 ~ 1279 之间\n");
            while (getchar() != '\n');
            break;
        }
        reclaim(addr);
        break;}

    case 3://重置位示图
        resetRandom();
        break;
    default:
        printf("输入错误, 请输入 0~3 之间的数字\n");
        break;
    }
}
return 0;
}

```

图 1.3 程序源码

五、运行截图

```
r00t@ub2:~$ ./main
已重置位示图 , free block: 645

=== 选择操作 ===
0. 显示位示图
1. 申请块
2. 回收块
3. 重置位示图
请输入操作: 0
0 0 0 0 1 1 1 1 0 1 0 1 0 1 0 0 0 0 0 0 1 1 1 0 1 1 1 1 0 1 0 1 1
1 0 0 0 1 1 1 1 0 0 0 0 1 1 1 1 1 1 0 1 0 1 0 1 0 1 1 1 0 1 1 1
1 1 1 0 0 1 0 0 1 0 1 0 1 0 1 1 1 1 0 1 0 0 0 0 0 0 0 0 0 1 1 1 0
0 0 0 0 1 0 0 0 1 1 0 0 1 1 1 0 1 1 1 1 0 0 0 0 0 0 0 0 0 1 1 1 1
1 1 1 1 0 1 1 1 1 0 1 0 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0 0 1
0 1 0 0 0 1 1 1 1 0 0 1 1 1 1 0 1 1 0 0 1 1 0 1 1 0 0 1 0 0 0 0 0
1 0 0 0 1 1 1 1 0 1 0 1 0 1 1 0 1 0 0 0 1 0 0 0 0 0 0 1 1 1 1 1 0
1 0 0 1 1 0 0 1 1 0 1 0 1 0 0 0 0 0 1 1 0 1 0 1 1 1 0 0 1 1 1 0
1 1 1 1 1 1 0 1 1 1 1 1 0 1 1 0 1 0 0 1 1 0 0 1 1 0 1 0 0 0 1 1
0 0 0 1 0 1 0 1 0 1 0 0 0 0 1 1 0 1 1 0 1 1 1 0 0 0 1 0 1 0 1 1
0 0 0 0 1 1 0 1 0 0 0 1 0 1 0 1 0 1 1 1 1 0 1 1 0 0 1 1 0 0 0 1
0 1 1 1 0 1 1 0 0 1 1 0 0 0 1 0 1 0 1 0 0 0 1 1 1 0 0 1 1 1 0 1
0 0 1 0 1 0 0 1 1 0 0 1 0 1 1 1 0 0 0 0 1 1 1 0 0 0 1 1 1 1 0 1
1 1 1 1 1 1 0 0 1 0 1 1 0 0 1 0 0 1 0 0 0 0 0 1 1 1 0 0 1 1 1 0 0
1 1 1 1 0 0 1 0 0 1 1 0 1 0 0 0 1 1 1 0 1 1 0 1 1 0 0 0 0 0 0 1
1 0 0 1 0 0 1 0 1 1 1 0 1 1 0 1 0 1 1 1 1 1 0 0 1 0 0 1 0 0 1 1
0 1 1 0 1 0 1 0 1 0 1 1 1 1 0 0 1 1 1 0 0 0 0 0 0 0 0 1 1 0 0 0 1
0 1 1 1 0 0 0 1 0 1 0 0 0 0 0 1 1 1 1 0 1 1 0 0 1 1 1 0 0 1 1 0
1 0 1 1 1 1 0 1 0 1 1 1 1 1 1 0 1 1 0 1 0 1 0 1 1 0 1 0 0 0
0 0 1 1 1 1 1 0 0 0 1 0 0 0 1 0 0 1 1 1 1 0 1 1 1 0 0 1 1 0 0 1
0 1 1 0 1 0 0 1 0 1 1 0 0 0 1 0 0 1 1 1 1 1 1 1 0 0 1 1 1 1 0 0
1 1 0 0 1 0 1 0 1 1 0 1 1 0 1 0 1 0 1 0 0 1 1 1 0 1 1 1 1 0
0 1 0 0 1 1 0 0 0 0 1 0 0 1 0 0 1 0 1 1 1 1 1 0 0 0 1 0 0 0 0 1
1 1 1 1 1 1 0 1 0 0 1 0 0 1 0 0 1 1 1 1 0 1 0 1 0 0 1 0 0 0 1 0
1 0 1 0 1 0 0 1 0 1 1 0 0 1 0 1 1 1 1 1 1 1 0 1 1 0 1 0 0 0 0 1
0 1 1 1 1 1 1 1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0 0 0 1 0 0 1 0 1 1
1 1 0 1 0 1 0 1 0 0 0 0 1 0 1 1 0 1 0 0 1 0 0 0 1 0 0 0 0 0 1 0
1 1 1 1 1 1 0 1 0 0 1 1 1 0 0 1 1 0 1 0 0 0 0 1 0 1 1 1 1 0 1 0
1 0 1 0 1 0 1 1 0 0 0 1 0 0 0 1 0 0 0 0 0 0 1 0 1 0 1 0 0 0 0 0
0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 1 0 0 1 0 1 1 1 0 1 0 1 0 1 1 0 1
1 0 1 1 0 1 0 1 1 0 0 1 0 0 0 1 1 0 1 0 1 0 0 0 1 1 0 0 1 0 1 0
1 1 0 1 0 0 0 1 0 0 0 0 1 1 1 0 1 1 0 0 1 0 0 0 0 1 0 1 1 0 1 0
1 1 0 1 1 0 0 1 1 0 0 0 1 1 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 0
```

图 1.4 程序执行显示位视图截图

```
=== 选择操作 ===
0. 显示位示图
1. 申请块
2. 回收块
3. 重置位示图
请输入操作: 1
请输入申请块数量: 15
申请|块地址 0 (柱面:0 磁头:0 扇区:0)
申请|块地址 1 (柱面:0 磁头:0 扇区:1)
申请|块地址 2 (柱面:0 磁头:0 扇区:2)
申请|块地址 3 (柱面:0 磁头:0 扇区:3)
申请|块地址 8 (柱面:0 磁头:1 扇区:0)
申请|块地址 10 (柱面:0 磁头:1 扇区:2)
申请|块地址 12 (柱面:0 磁头:1 扇区:4)
申请|块地址 14 (柱面:0 磁头:1 扇区:6)
申请|块地址 15 (柱面:0 磁头:1 扇区:7)
申请|块地址 16 (柱面:0 磁头:2 扇区:0)
申请|块地址 17 (柱面:0 磁头:2 扇区:1)
申请|块地址 18 (柱面:0 磁头:2 扇区:2)
申请|块地址 22 (柱面:0 磁头:2 扇区:6)
申请|块地址 27 (柱面:0 磁头:3 扇区:3)
申请|块地址 29 (柱面:0 磁头:3 扇区:5)
申请完成, free block: 630
```

图 1.5 程序执行分配磁盘块成功截图

```
=== 选择操作 ===
0. 显示位示图
1. 申请块
2. 回收块
3. 重置位示图
请输入操作: 1
请输入申请块数量: 2222
出错: 申请块数 > 空闲块数
```

图 1.6 程序执行分配磁盘块失败截图-输入块数大于空闲块

```
=== 选择操作 ===
0. 显示位示图
1. 申请块
2. 回收块
3. 重置位示图
请输入操作: 1
请输入申请块数量: -25
出错: 申请块数非正整数
```

图 1.7 程序执行分配磁盘块失败截图-输入块数非正整数

```
=== 选择操作 ===
0. 显示位示图
1. 申请块
2. 回收块
3. 重置位示图
请输入操作: 2
请输入块地址: 0
回收|块地址 0 (柱面:0 磁头:0 扇区:0)
free block: 602
```

图 1.8 程序执行回收磁盘块成功截图


```
=== 选择操作 ===
0. 显示位示图
1. 申请块
2. 回收块
3. 重置位示图
请输入操作：2
请输入块地址：0
出错：该磁盘块为空闲块
```

图 1.9 程序执行分配磁盘块失败截图-输入块地址为空闲块

```
=== 选择操作 ===
0. 显示位示图
1. 申请块
2. 回收块
3. 重置位示图
请输入操作：2
请输入块地址：-28
出错：块地址非法，应在0 ~ 1279 之间
```

图 1.10 程序执行分配磁盘块失败截图-输入块地址非法

小标题 2：在 Linux 操作系统上，简单地实现一个基于位示图的文件系统

一、题目要求

- (1) 为了方便文件系统的编写和测试，最好在 VMWare 虚拟机上实现。
- (2) Linux 操作系统一般都会自带 ramdisk 设备(/dev/ram)，它能把一部分内存模拟成一个磁盘。所以，在实验的过程中可以使用 ramdisk 作为块设备，而不需要专门为该文件系统直接创建一个磁盘分区。
- (3) 文件系统的实现推荐参考 Linux 内核源代码中 minix 文件系统的实现方式。它的实现在(Linux 内核源码目录/fs/minix/)目录中。

二、需求分析

在 Linux 上实现一个简单的文件系统，其核心功能依赖位示图来管理磁盘块。实现的基本的功能包括：新增、删除、重命名、读写、拷贝目录或文件。另外还需完成初始化文件系统、显示目录内容功能。

三、实验环境

Ubuntu 14.04 LTS

gcc version 4.8.4

编译标准：C99

四、源码注解

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>

#define BLOCK_SIZE 512 // 定义每个磁盘块的大小为 512 byte
#define TOTAL_BLOCKS 1280 // 定义文件系统总共包含 1280 个磁盘块
#define FS_SIZE (BLOCK_SIZE * TOTAL_BLOCKS) //定义文件系统的总大小
#define MAX_FILENAME 28 // 定义文件/目录名最大长度为 28 字符
#define MAX_FILES 128 // 定义最大文件/目录的总数为 128
#define DIR_ENTRY_SIZE sizeof(struct dir_entry) // 定义目录项结构体的大小

const char *ramdisk = "/dev/ram0"; // 定义虚拟磁盘设备文件路径

struct super_block { // 超级块
```

```

int total_blocks;    // 总块数
int free_blocks;     // 空闲块数
int bitmap_offset;   // 位图在磁盘上的起始块号
int root_dir_offset; // 根目录在磁盘上的起始块号
};

struct dir_entry { // 目录项
    char name[MAX_FILENAME]; // 文件/目录名
    int is_directory;         // 标志位：0 为文件，1 为目录
    int start_block;          // 文件/目录数据在磁盘上的起始块号
    int size;                 // 文件/目录占用的块数 [然而在当前实现中，所有文件大小都为 1 块!!!!!!]
};

struct super_block sb; // 初始化超级块实例，用于在内存中维护文件系统的超级块信息

unsigned char bitmap[TOTAL_BLOCKS]; // 初始化位图阵列变量 bitmap，用于跟踪每个磁盘块的使用
情况，0 为空闲，1 为已使用

struct dir_entry root_dir[MAX_FILES]; // 初始化根目录表变量 root_dir，在内存中维护根目录下的所有
文件和目录的目录项信息

void write_block(int fd, int block_num, void *buf) {
    /*
    * 写入数据到指定块
    * 参数：
    * fd: 文件描述符
    * block_num: 要写入的块的逻辑块号
    * buf: 待写入数据的缓冲区指针
    */

    lseek(fd, block_num * BLOCK_SIZE, SEEK_SET);
    // 使用 lseek 定位到文件描述符 fd 中 block_num 块的起始位置
    // BLOCK_SIZE * block_num 得到该块在磁盘上的字节偏移量
    write(fd, buf, BLOCK_SIZE);
    // 从 buf 缓冲区写入 BLOCK_SIZE 大小的数据到当前文件描述符位置
}

void read_block(int fd, int block_num, void *buf) {
    /*
    * 从指定块读取数据
    * 参数：
    * fd: 文件描述符
    * block_num: 要读取的块的逻辑块号
    * buf: 存放读取数据的缓冲区指针
    */

    lseek(fd, block_num * BLOCK_SIZE, SEEK_SET);
    // 使用 lseek 定位到文件描述符 fd 中 block_num 块的起始位置

    read(fd, buf, BLOCK_SIZE);
    // 从当前文件描述符位置读取 BLOCK_SIZE 大小的数据到 buf 缓冲区
}

```

```

}

int allocate_block() {
    /*
    *分配一个空闲块
    *返回值：
    *成功：块号
    *失败：-1
    */
    for (int i = 0; i < TOTAL_BLOCKS; i++) { //遍历位图查找块
        if (bitmap[i] == 0) { // 找到一个空闲块
            bitmap[i] = 1; // 将该块标记为已使用
            sb.free_blocks--; // 更新超级块中空闲块的数量
            return i; // 返回分配到的块号
        }
    }
    return -1; // 没有空闲块，返回失败
}

void free_block(int block_num) {
    /*
    *释放一个块
    *参数：
    *block_num: 要释放的块的逻辑块号
    */

    if (bitmap[block_num]) { // 检查该块是否已被占用，如是，释放
        bitmap[block_num] = 0; // 标记该块为空闲
        sb.free_blocks++; // 更新空闲块数量
    }
}

void save_metadata(int fd) {
    /*将内存中的元数据（超级块、位图、根目录）写入磁盘
    *参数：
    *fd: 文件描述符
    */
    lseek(fd, 0, SEEK_SET); //定位磁盘起始位置
    write(fd, &sb, sizeof(sb)); // 写入超级块数据到磁盘
    write(fd, bitmap, sizeof(bitmap)); //写入位图数据到磁盘
    lseek(fd, sb.root_dir_offset * BLOCK_SIZE, SEEK_SET); // 保存根目录
    write(fd, root_dir, sizeof(root_dir)); //写入根目录数据到磁盘
}

void load_metadata(int fd) {
    /*
    *从磁盘加载元数据到内存
    *参数:
    *fd:文件描述符
    */

    lseek(fd, 0, SEEK_SET); // 定位磁盘起始位置
    read(fd, &sb, sizeof(sb)); // 从磁盘读取超级块数据到内存中 sb 变量
    read(fd, bitmap, sizeof(bitmap)); // 从磁盘读取位图数据到内存中的 bitmap 阵列

```

```

lseek(fd, sb.root_dir_offset * BLOCK_SIZE, SEEK_SET); //读取根目录
read(fd, root_dir, sizeof(root_dir)); // 从磁盘读取根目录数据到内存中的 root_dir 阵列
}

void format_fs(int fd) {
    /*
    *格式化文件系统
    *参数:
    *fd:文件描述符
    */
    sb.total_blocks = TOTAL_BLOCKS; // 设置总块数

    sb.root_dir_offset = 20; //目录表偏移到第 20 块，避免覆盖数据块

    memset(bitmap, 0, sizeof(bitmap)); // 初始化位图，标记所有块为空闲

    //start 标记 0~(root_dir 占用块)为已用
    int metadata_blocks = sb.root_dir_offset + (sizeof(root_dir) + BLOCK_SIZE - 1) / BLOCK_SIZE;
    for (int i = 0; i < metadata_blocks; i++) {
        bitmap[i] = 1;
    }
    //end 标记 0~(root_dir 占用块)为已用

    memset(root_dir, 0, sizeof(root_dir)); // 初始化根目录

    save_metadata(fd); //将初始化后的元数据写入到磁盘
}

int find_file(const char *name) {
    /*
    *在根目录中查找文件或目录
    *参数：
    *name: 要查找的文件或目录的名称
    *返回值：
    *成功：目录项的索引
    *失败：-1
    */
    for (int i = 0; i < MAX_FILES; i++) {
        if (strcmp(root_dir[i].name, name) == 0) { // 使用 strcmp 比较名称，如果匹配则返回当前索引
            return i;
        }
    }
    return -1; // 返回失败
}

int find_empty_dir_entry() {
    /*
    *查找一个空闲的目录项位置
    *返回值：
    *成功：空目录项的索引

```

```

*失败：-1
*/
for (int i = 0; i < MAX_FILES; i++) { //遍历根目录表
    if (root_dir[i].name[0] == '\0') { // 检查名称的第一个字符是否为空
        return i; //返回空目录项的索引
    }
}
return -1; // 目录项已满，返回失败
}

void create_entry(int fd, const char *name, int is_dir, int size) {
    /*
    *
    *创建文件或目录
    *参数：
    *fd: 文件描述符
    *name: 文件/目录的名称
    *is_dir: 标志位，0 为文件，1 为目录
    *size: 文件/目录占用的块数 [当前实现仅能为 1！！！！]
    */

    if (find_file(name) != -1) { // 检查是否存在同名文件或目录
        printf("已存在该名称的文件或目录\n");
        return;
    }
    int idx = find_empty_dir_entry(); // 查找一个空闲的目录项，保存空目录项的索引

    if (idx == -1) { //判断是否找不到空目录
        printf("目录项已满\n");
        return;
    }

    int start_block = allocate_block(); // 分配一个空闲块给新的文件或目录，保存块号

    if (start_block == -1) { //判断是否没有空块
        printf("磁盘空间不足\n");
        return;
    }

    //start 初始化新的目录项
    strncpy(root_dir[idx].name, name, MAX_FILENAME - 1); // 复制名称，确保以 '\0' 结尾
    root_dir[idx].name[MAX_FILENAME - 1] = '\0'; // 强制 null 终止
    root_dir[idx].is_directory = is_dir; // 设置类型 (文件或目录)
    root_dir[idx].start_block = start_block; // 设置起始块号
    root_dir[idx].size = size; // 设置占用块数
    //end 初始化新的目录项

    save_metadata(fd); // 将更新后的元数据写入磁盘
    printf("创建%s '%s' 成功\n", is_dir ? "目录" : "文件", name);
}

void delete_entry(int fd, const char *name) {

```

```

/*
 *删除文件或目录并释放其占用的块
 *参数：
 *fd: 文件描述符
 *name: 要删除的文件或目录的名称
 */

int idx = find_file(name); // 查找要删除的文件/目录的索引

if (idx == -1) { //判断文件/目录是否存在
    printf("找不到文件或目录\n");
    return;
}
for (int i = 0; i < root_dir[idx].size; i++) {
    free_block(root_dir[idx].start_block + i); // 释放该文件/目录占用的所有块
}

memset(&root_dir[idx], 0, sizeof(struct dir_entry)); // 清除内存中的目录项信息

save_metadata(fd); // 将更新后的元数据写入磁盘

printf("已删除 '%s'\n", name);
}

//

void rename_entry(int fd, const char *old_name, const char *new_name) {
/*
 *重命名文件或目录
 *参数：
 *fd: 文件描述符
 *old_name: 旧名称
 *new_name: 新名称
 */

int idx = find_file(old_name); // 查找旧名称文件/目录对应的目录项索引

if (idx == -1) { //判断旧名称文件/目录是否存在
    printf("找不到文件或目录\n");
    return;
}

strncpy(root_dir[idx].name, new_name, MAX_FILENAME - 1); //更新目录项的名称
root_dir[idx].name[MAX_FILENAME - 1] = '\0'; //强制 null 终止
save_metadata(fd); // 将更新后的元数据写入磁盘
printf("重命名 '%s' 为 '%s' 成功\n", old_name, new_name);
}

void list_dir() {
/*
 *显示当前根目录中的所有文件和目录
 */
printf("当前目录内容 : \n");

```

```

for (int i = 0; i < MAX_FILES; i++) {
    if (root_dir[i].name[0] != '\0') { // 检查目录项是否被占用
        printf("%s\t%s\t块号: %d 大小: %d\n",
            root_dir[i].is_directory ? "[目录]" : "[文件]", //判断是目录还是文件
            root_dir[i].name, // 名称
            root_dir[i].start_block, // 起始块号
            root_dir[i].size); // 占用大小
    }
}
}

```

```

void write_file(int fd, const char *name, const char *data) {
    /*
    *向指定文件写入数据
    *参数：
    *fd:文件描述符
    *name:文件名
    *data:数据
    */
    int idx = find_file(name); // 查找文件索引
    if (idx == -1 ) { // 判断文件是否存在
        printf("找不到该文件或它是目录\n");
        return;
    }
    if ( root_dir[idx].is_directory) { //判断文件名是否为目录
        printf("名称为目录\n");
        return;
    }
    char buf[BLOCK_SIZE] = {0}; // 初始化缓冲区为 0，防止残留脏数据
    strncpy(buf, data, BLOCK_SIZE - 1); // 拷贝数据到缓冲区，确保不溢出
    write_block(fd, root_dir[idx].start_block, buf); // 将数据写入文件的起始块

    printf("写入完成\n");
}

```

```

void read_file(int fd, const char *name) {
    /*
    *从指定文件读取数据
    *参数：
    *fd:文件描述符
    *name:文件名
    */

    int idx = find_file(name); // 查找文件索引

    if (idx == -1 ) { // 判断文件是否存在
        printf("找不到该文件或它是目录\n");
        return;
    }
    if ( root_dir[idx].is_directory) { //判断文件名是否为目录
        printf("名称为目录\n");
        return;
    }
}

```



```

char buf[BLOCK_SIZE] = {0}; //创建一个缓冲区用于存放读取的数，并初始化为 0
read_block(fd, root_dir[idx].start_block, buf); //从文件起始块读取数据
printf("文件内容 : \n%s\n", buf);
}

void copy_entry(int fd, const char *src, const char *dst) {
/*
*复制文件或目录
*参数：
*fd: 文件描述符
*src: 源文件/目录的名称
*dst: 目标文件/目录的名称
*/
int idx = find_file(src); // 查找源文件/目录在文件系统中的索引
if (idx == -1) { // 判断源文件/目录是否存在
    printf("源文件或目录不存在\n");
    return;
}

char buf[BLOCK_SIZE] = {0}; //初始化缓存区
read_block(fd, root_dir[idx].start_block, buf); //读取源文件的数据

int new_start_block = allocate_block(); // 分配一个空闲块给新文件/目录
if (new_start_block == -1) { //判断是否分配失败，如是，说明磁盘空间不足
    printf("磁盘空间不足，复制失败\n");
    return;
}

int new_idx = find_empty_dir_entry();
if (new_idx == -1) { // 如果没有空闲目录项，如是，说明目录项已满
    free_block(new_start_block); // 释放已分配的块
    printf("目录项已满，复制失败\n");
    return;
}

// start 写入新的目录项
strncpy(root_dir[new_idx].name, dst, MAX_FILENAME - 1);
root_dir[new_idx].name[MAX_FILENAME - 1] = '\0'; // 确保字符串以空字符结尾
root_dir[new_idx].is_directory = root_dir[idx].is_directory; // 复制是否为目录的标志
root_dir[new_idx].start_block = new_start_block; // 设置新文件/目录的起始块号
root_dir[new_idx].size = 1; // 复制文件大小为 1 块 [对于当前实现，文件大小固定为 1 块！！]
//end 写入新的目录项

write_block(fd, new_start_block, buf); //写入数据到新文件块

save_metadata(fd); //保存新的目录结构和位图

printf("已复制 '%s' 为 '%s'\n", src, dst);
}

```

```

int main() {
    int fd = open(ramdisk, O_RDWR); //以读写模式打开虚拟磁盘设备文件

    if (fd < 0) {
        perror("打开 ramdisk 失败"); //如果打开失败，打印错误信息
        return 1; //返回错误码，结束运行
    }

    char name[128], name2[128], data[BLOCK_SIZE]; //用于记录用户输入文件/目录名和数据

    load_metadata(fd); // 加载元数据

    int choice; //记录用户输入

    while (1) {
        printf("\n=== 功能选单 ===\n");
        printf("0. 格式化文件系统\n");
        printf("1. 创建文件\n");
        printf("2. 创建目录\n");
        printf("3. 删除文件/目录\n");
        printf("4. 重命名\n");
        printf("5. 复制文件/目录\n");
        printf("6. 写入文件\n");
        printf("7. 读取文件\n");
        printf("8. 显示目录内容\n");
        printf("9. 退出\n> ");
        scanf("%d", &choice); // 读取用户选择
        getchar(); // 吸收 scanf 留下的换行符

        switch (choice) { //根据用户选择执行相应操作
            case 0:
                printf("确定执行格式化吗?:[y/N] ");
                scanf("%c", (char *)&choice); //用户输入是否确定格式化
                while ((name[0] = getchar()) != '\n' && name[0] != EOF); //清空输入缓冲区，防止死循环
                if(choice=='y') { //判断用户是否确定执行格式化
                    format_fs(fd); //格式化文件系统
                    break;
                }

                printf("未执行格式化\n");
                break;
            case 1:
                printf("输入文件名: ");
                fgets(name, sizeof(name), stdin); //读取文件名
                name[strcspn(name, "\n")] = 0; // 去除文件名末尾的换行符
                create_entry(fd, name, 0, 1); //创建文件(is_dir=0)
                break;
            case 2:
                printf("输入目录名: ");
                fgets(name, sizeof(name), stdin); // 读取目录名
                name[strcspn(name, "\n")] = 0; // 去除目录名末尾的换行符
                create_entry(fd, name, 1, 1); // 创建目录 (is_dir =1)
                break;
        }
    }
}

```

```

case 3:
    printf("输入名称: ");
    fgets(name, sizeof(name), stdin); // 读取要删除的名称
    name[strcspn(name, "\n")] = 0;
    delete_entry(fd, name); // 删除文件或目录
    break;
case 4:
    printf("输入旧名称: ");
    fgets(name, sizeof(name), stdin); // 读取旧名称
    name[strcspn(name, "\n")] = 0;
    printf("输入新名称: ");
    fgets(name2, sizeof(name2), stdin); // 读取新名称
    name2[strcspn(name2, "\n")] = 0;
    rename_entry(fd, name, name2); // 执行重命名
    break;
case 5:
    printf("输入源名称: ");
    fgets(name, sizeof(name), stdin); // 读取源名称
    name[strcspn(name, "\n")] = 0;
    printf("输入目标名称: ");
    fgets(name2, sizeof(name2), stdin); // 读取目标名称
    name2[strcspn(name2, "\n")] = 0;
    copy_entry(fd, name, name2); // 复制
    break;
case 6:
    printf("输入文件名: ");
    fgets(name, sizeof(name), stdin); // 读取文件名
    name[strcspn(name, "\n")] = 0;
    printf("输入写入数据: ");
    fgets(data, sizeof(data), stdin); // 读取要写入的数据
    data[strcspn(data, "\n")] = 0;
    write_file(fd, name, data); // 写入文件
    break;
case 7:
    printf("输入文件名: ");
    fgets(name, sizeof(name), stdin); // 读取文件名
    name[strcspn(name, "\n")] = 0;
    read_file(fd, name); // 读取文件
    break;
case 8:
    list_dir(); // 显示目录内容
    break;
case 9:
    close(fd); // 关闭磁盘文件描述符
    return 0; // 退出程序
default:
    printf("无效选项\n"); // 处理无效输入
}
}
}

```

图 2.1 程序源码

五、运行记录

```
sudo su
modprobe brd rd_nr=1 rd_size=640
dd if=/dev/zero of=/dev/ram0 bs=512 count=1280
```

图 2.2.1 创建及初始化 ramdisk shell 命令

```
root@ub2:/home/r00t# sudo su
root@ub2:/home/r00t# modprobe brd rd_nr=1 rd_size=640
root@ub2:/home/r00t# dd if=/dev/zero of=/dev/ram0 bs=512 count=1280
1280+0 records in
1280+0 records out
655360 bytes (655 kB) copied, 0.0023361 s, 281 MB/s
```

图 2.2.2 创建及初始化 ramdisk shell 执行记录截图

准备工作：
本系统未自带 ramdisk 设备 (/dev/ram)，使用图 2.2.1 命令创建及初始化 ramdisk，运行结果如图 2.2.2 所示。

功能展示：

```
=== 功能选单 ===
0. 格式化文件系统
1. 创建文件
2. 创建目录
3. 删除文件/目录
4. 重命名
5. 复制文件/目录
6. 写入文件
7. 读取文件
8. 显示目录内容
9. 退出
> 8
当前目录内容：
[文件] abc.txt 块号：1 大小：1

=== 功能选单 ===
0. 格式化文件系统
1. 创建文件
2. 创建目录
3. 删除文件/目录
4. 重命名
5. 复制文件/目录
6. 写入文件
7. 读取文件
8. 显示目录内容
9. 退出
> 0
确定执行格式化吗?:[y/N] y
文件系统已格式化

=== 功能选单 ===
0. 格式化文件系统
1. 创建文件
2. 创建目录
3. 删除文件/目录
4. 重命名
5. 复制文件/目录
6. 写入文件
7. 读取文件
8. 显示目录内容
9. 退出
> 8
当前目录内容：
```

图 2.3 实现格式化截图

图 2.3 展示了，ramdisk 内原本有内容，执行格式化后，内容被清除。

```
=== 功能选单 ===
0. 格式化文件系统
1. 创建文件
2. 创建目录
3. 删除文件/目录
4. 重命名
5. 复制文件/目录
6. 写入文件
7. 读取文件
8. 显示目录内容
9. 退出
> 8
当前目录内容：
[目录]  f1      块号：0 大小：1
[文件]  abc.xxxx 块号：1 大小：1
[文件]  a.x      块号：2 大小：1
```

```
=== 功能选单 ===
0. 格式化文件系统
1. 创建文件
2. 创建目录
3. 删除文件/目录
4. 重命名
5. 复制文件/目录
6. 写入文件
7. 读取文件
8. 显示目录内容
9. 退出
> 8
当前目录内容：
```

图 2.4.1 获取目录内容截图（有内容） 图 2.4.2 获取目录内容截图（无内容）

图 2.4.1 和图 2.4.2 展示了，ramdisk 内原本有内容时与无内容时获取目录内容函数执行结果。

```
=== 功能选单 ===
0. 格式化文件系统
1. 创建文件
2. 创建目录
3. 删除文件/目录
4. 重命名
5. 复制文件/目录
6. 写入文件
7. 读取文件
8. 显示目录内容
9. 退出
> 8
当前目录内容：

=== 功能选单 ===
0. 格式化文件系统
1. 创建文件
2. 创建目录
3. 删除文件/目录
4. 重命名
5. 复制文件/目录
6. 写入文件
7. 读取文件
8. 显示目录内容
9. 退出
> 1
输入文件名：abcd.txt
创建文件 'abcd.txt' 成功

=== 功能选单 ===
0. 格式化文件系统
1. 创建文件
2. 创建目录
3. 删除文件/目录
4. 重命名
5. 复制文件/目录
6. 写入文件
7. 读取文件
8. 显示目录内容
9. 退出
> 2
输入目录名：img
创建目录 'img' 成功

=== 功能选单 ===
0. 格式化文件系统
1. 创建文件
2. 创建目录
3. 删除文件/目录
4. 重命名
5. 复制文件/目录
6. 写入文件
7. 读取文件
8. 显示目录内容
9. 退出
> 8
当前目录内容：
[文件]  abcd.txt      块号：1 大小：1
[目录]  img          块号：2 大小：1
```

图 2.5 实现创建文件及目录截图

图 2.5 展示了，ramdisk 创建文件及创建目录后，获取目录内容结果。

```
=== 功能选单 ===
0. 格式化文件系统
1. 创建文件
2. 创建目录
3. 删除文件/目录
4. 重命名
5. 复制文件/目录
6. 写入文件
7. 读取文件
8. 显示目录内容
9. 退出
> 5
输入源名称: abcd.txt
输入目标名称: xyz.txt
已复制 'abcd.txt' 为 'xyz.txt'

=== 功能选单 ===
0. 格式化文件系统
1. 创建文件
2. 创建目录
3. 删除文件/目录
4. 重命名
5. 复制文件/目录
6. 写入文件
7. 读取文件
8. 显示目录内容
9. 退出
> 5
输入源名称: img
输入目标名称: photo
已复制 'img' 为 'photo'

=== 功能选单 ===
0. 格式化文件系统
1. 创建文件
2. 创建目录
3. 删除文件/目录
4. 重命名
5. 复制文件/目录
6. 写入文件
7. 读取文件
8. 显示目录内容
9. 退出
> 8
当前目录内容：
[文件]  abcd.txt      块号: 1 大小: 1
[目录]  img          块号: 2 大小: 1
[文件]  xyz.txt      块号: 3 大小: 1
[目录]  photo        块号: 4 大小: 1
```

图 2.6.1 实现复制文件/目录截图

图 2.6.1 展示了，复制文件及创建目录后，获取目录内容结果。
图 2.6.2 展示了，复制文件后文件内容成功被复制。

```
=== 功能选单 ===
0. 格式化文件系统
1. 创建文件
2. 创建目录
3. 删除文件/目录
4. 重命名
5. 复制文件/目录
6. 写入文件
7. 读取文件
8. 显示目录内容
9. 退出
> 5
输入源名称: a
输入目标名称: b
已复制 'a' 为 'b'

=== 功能选单 ===
0. 格式化文件系统
1. 创建文件
2. 创建目录
3. 删除文件/目录
4. 重命名
5. 复制文件/目录
6. 写入文件
7. 读取文件
8. 显示目录内容
9. 退出
> 7
输入文件名: a
文件内容：
abc

=== 功能选单 ===
0. 格式化文件系统
1. 创建文件
2. 创建目录
3. 删除文件/目录
4. 重命名
5. 复制文件/目录
6. 写入文件
7. 读取文件
8. 显示目录内容
9. 退出
> 7
输入文件名: b
文件内容：
abc
```

图 2.6.2 实现复制文件截图

```
=== 功能选单 ===
0. 格式化文件系统
1. 创建文件
2. 创建目录
3. 删除文件/目录
4. 重命名
5. 复制文件/目录
6. 写入文件
7. 读取文件
8. 显示目录内容
9. 退出
> 6
输入文件名: SCUT.txt
输入写入数据: 华南理工大学
写入完成
```

图 2.7.1 写入文件截图

```
=== 功能选单 ===
0. 格式化文件系统
1. 创建文件
2. 创建目录
3. 删除文件/目录
4. 重命名
5. 复制文件/目录
6. 写入文件
7. 读取文件
8. 显示目录内容
9. 退出
> 7
输入文件名: SCUT.txt
文件内容：
华南理工大学
```

图 2.7.2 读取文件截图

图 2.7.1 和图 2.7.2 展示了，写入文件与读取文件结果截图。

```
=== 功能选单 ===
0. 格式化文件系统
1. 创建文件
2. 创建目录
3. 删除文件/目录
4. 重命名
5. 复制文件/目录
6. 写入文件
7. 读取文件
8. 显示目录内容
9. 退出
> 8
当前目录内容：
[文件] a      块号：30 大小：1
[文件] aa     块号：31 大小：1
[文件] aaa    块号：32 大小：1

=== 功能选单 ===
0. 格式化文件系统
1. 创建文件
2. 创建目录
3. 删除文件/目录
4. 重命名
5. 复制文件/目录
6. 写入文件
7. 读取文件
8. 显示目录内容
9. 退出
> 3
输入名称：aa
已删除 'aa'

=== 功能选单 ===
0. 格式化文件系统
1. 创建文件
2. 创建目录
3. 删除文件/目录
4. 重命名
5. 复制文件/目录
6. 写入文件
7. 读取文件
8. 显示目录内容
9. 退出
> 8
当前目录内容：
[文件] a      块号：30 大小：1
[文件] aaa    块号：32 大小：1
```

图 2.8.1 文件删除截图

```
=== 功能选单 ===
0. 格式化文件系统
1. 创建文件
2. 创建目录
3. 删除文件/目录
4. 重命名
5. 复制文件/目录
6. 写入文件
7. 读取文件
8. 显示目录内容
9. 退出
> 8
当前目录内容：
[文件] a      块号：30 大小：1
[目录] photo  块号：31 大小：1

=== 功能选单 ===
0. 格式化文件系统
1. 创建文件
2. 创建目录
3. 删除文件/目录
4. 重命名
5. 复制文件/目录
6. 写入文件
7. 读取文件
8. 显示目录内容
9. 退出
> 3
输入名称：photo
已删除 'photo'

=== 功能选单 ===
0. 格式化文件系统
1. 创建文件
2. 创建目录
3. 删除文件/目录
4. 重命名
5. 复制文件/目录
6. 写入文件
7. 读取文件
8. 显示目录内容
9. 退出
> 8
当前目录内容：
[文件] a      块号：30 大小：1
```

图 2.8.2 目录删除截图

图 2.8.1 和图 2.8.2 展示了，文件或目录删除功能结果截图。